МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Кафедра «ИТ»

Лабораторная работа № 5
по дисциплине «Введение в информационные технологии»
по теме: «Объектно-ориентированное программирование в Python»

Выполнил студент группы: БВТ
Номер группы 2604
ФИО Молчанов Матвей Владимирович
Проверил: Мосева. М. С.
Павликов А. Е.

Москва 2026

Оглавление

Введение ..... **Ошибка! Закладка не определена.**

Выполнение 4

Выводы 9

Цель и задачи работы:

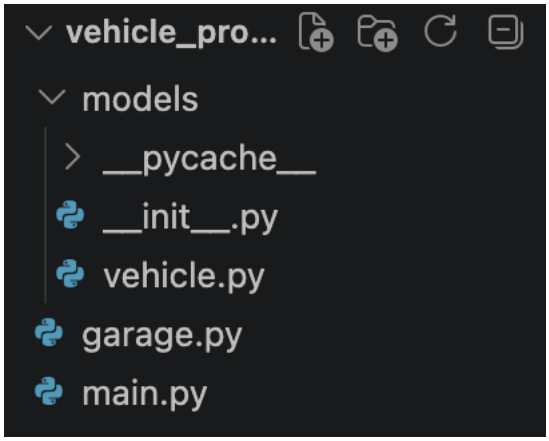
Целью данной лабораторной работы является: изучить основные принципы объектно-ориентированного программирования в Python; научиться создавать классы и объекты, определять атрибуты и методы, использовать конструктор класса, применять инкапсуляцию, наследование, абстракцию и полиморфизм, а также переопределять методы родительских классов.

Задачи: В ходе выполнения лабораторной работы необходимо разработать небольшой объектно-ориентированный проект управления транспортными средствами, последовательно применяя изучаемые возможности языка Python.

Выполнение

Задание 1.

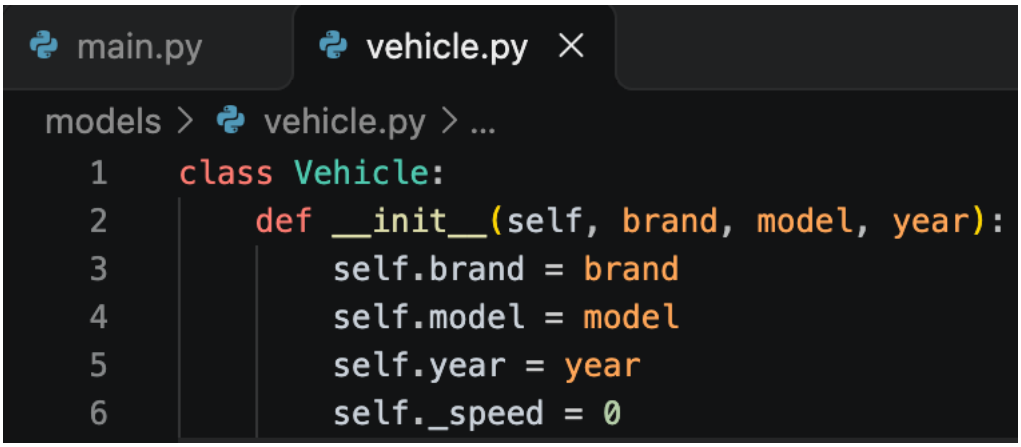
Создал папки и файлы как сказано в задании.



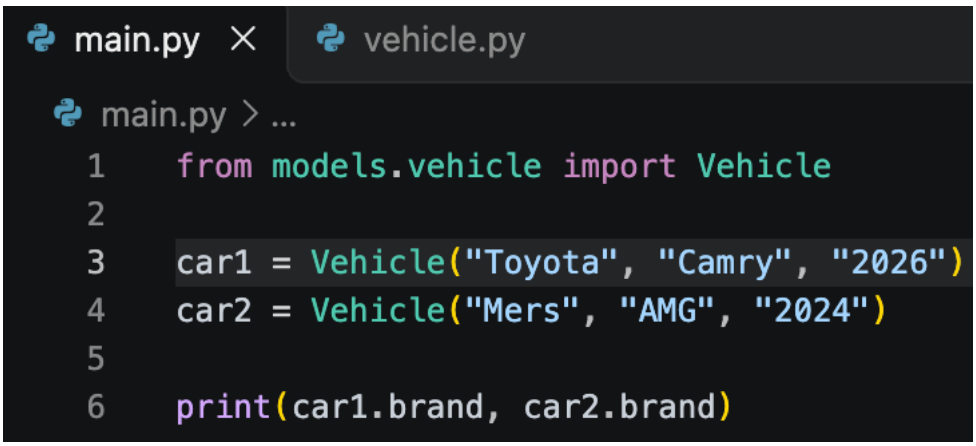
Картинка 1. Структура проекта.

Задание 2.

Создал класс Vehical, реализовал конструктор для бренда модели и года машины, добавил атрибут _speed со значением 0. Потом в main создал 2 обьекта Vehicle и вывел значения атрибутов, они имеют независиморе состояние.



Картинка 2. Конструктор.



Картинка 3. Vehical в main.

Задание 3.

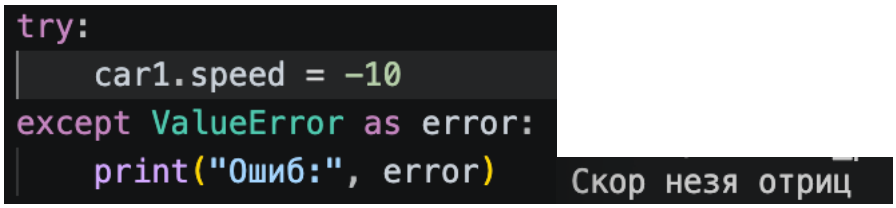
Добавил методы accelerate для увеличения скорости, brake для уменьшения и сделал ограничение чтобы ниже 0 не опускалось и stop для остановки.



Картинка 4. Работа методов в main.

Задание 4.

Сделал внутреннее поле _speed, настоящее хранилище значения скорости в __init__, property – дверь на чтение и сеттер на запись.



Картинка 5. Отрицательная скорость и ошибка.

Задание 5.

Добавил __str__ для того чтобы информацию о машинах можно было выводить без get info а просто через принт кар1.

```
car1.accelerate(50)
car1.brake(20)
car2.accelerate(100)
car2.stop(0)
print(car1, car2)
```

```
Toyota Camry, 2026, 30 Mers AMG, 2024, 0
```

Картинка 6. Метод __str__

Задание 6.

Создал файл car.py и класс Car и добавил атрибут двери, инициализировал общую часть обьекта через супер и добавил количество дверей

```
car = Car("Toyota", "Camry", 2024, 4)
print(car.get_info())
```

```
Toyota Camry, 2024, скор 0, дверей: 4
```

Картинка 7. Добавление количества дверей.

Задние 7.

Создал файлы и классы трак и мотосайкл с доп атрибутами.

```
Toyota Camry, 2024, скор 0, дверей: 4
mer A80, 2020, скор 0, грузоп: 400
BMW X1000, 2025, скор 0, без коляски
```

Картинка 8. Трак и мотосайкл с доп атрибутами.

Задание 8.

Теперь класс наследуется от ABC и добавлен метод move(), реализовал его в классах. И нельзя создавать классы пока не реализован абстрактный метод.

```
TypeError: Can't instantiate abstract class Vehicle without an implementation
for abstract method 'move'
```

Картинка 9. Попытка создать вехикл без реализации мува.

```
try:
    Vehicle("X", "Y", 2020)
except TypeError as error:
    print("Нельзя", error)
```

Картинка 10. Код попытки создания.

Задание 9.

Создал список и реализуем move().

```
Автомобиль едет по дороге
Toyota Camry, 2024, скор 0, дверей: 4
truck едет по дороге
mer A80, 2020, скор 0, грузоп: 400
моцик едет по дороге
BMW X1000, 2025, скор 0, без коляски
```

Картинка 11. Вывод с реализацией move().

Задание 10.

Создал гараж и сделал добавление туда разных классов.

```
Toyota Camry, 2024, 0
mer A80, 2020, 0
BMW X1000, 2025, 0
Toyota Camry, 2024, 0
```

```
garage = Garage()
for vehicle in vehicles:
    garage.add_vehicle(vehicle)
garage.show_all()
for vehicle in garage.find_by_brand("Toyota"):
    print(vehicle)
```

Картинка 12. Вывод и код гаража.

Задание 11.

Завернул все в функцию main() и добавил проверку на основной файл.

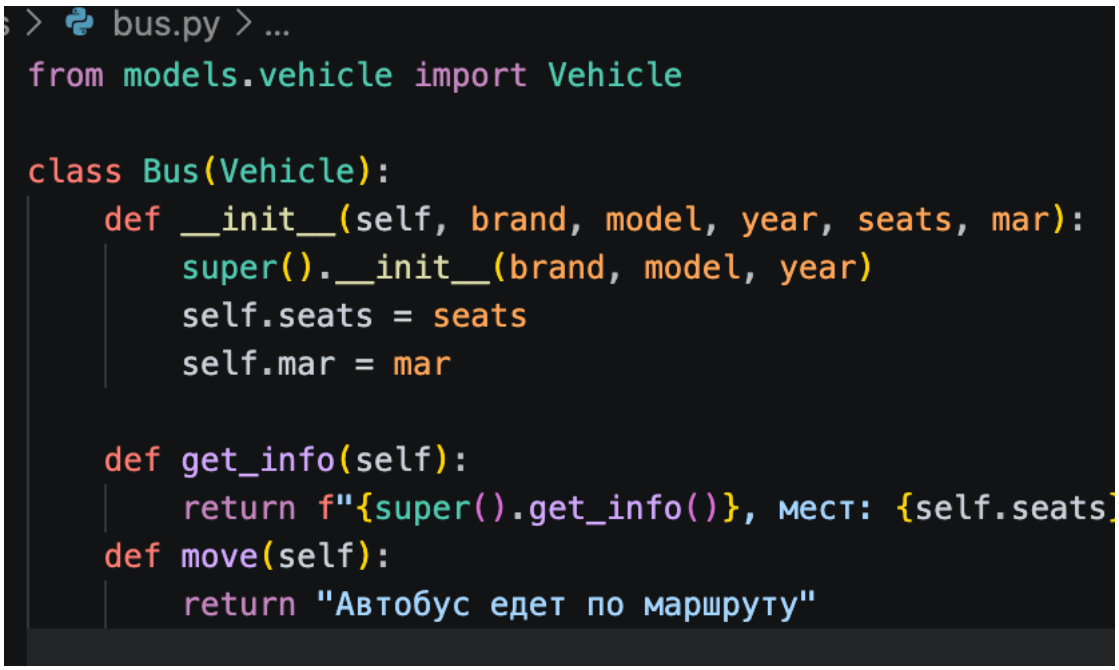
```
def main():
    car = Car(
```

```
if __name__ == "__main__":
    main()
```

Картинка 13. Main() and проверка.

Задание 12.

Создал свой класс, добавил в него атрибуты, добавляется в гараж без именения внутренней логики.



Картинка 14. Новый класс с автобусом.

Вывод

В ходе лабораторной работы мы получили навыки объектно-ориентированного программирования в Python: научились создавать классы и объекты, применять инкапсуляцию, наследование, полиморфизм и абстракцию.

В результате был разработан проект vehicle_project, демонстрирующий все основные принципы ООП на примере системы управления транспортными средствами.